
H.SYN.EMA CODEC

DESIGN & ARCHITECTURE · THE ENCODER AND DECODER

a codec that compiles decisions, not pixels

encode a production into an instruction stream; decode it back into a film, or its proof

ABSTRACT

A conventional video codec spends its intelligence hiding redundancy between pixels that already exist. The H.SYN.EMA codec has the opposite job: the pixels do not exist yet, and the codec's task is to store the *decisions* that will produce them and to make those decisions reproduce. The encoder is therefore a scheduler over a generative model and a multiplexer of evidence — it segments a production into coding units (intra keyframes and predicted deltas), captures the provenance of every synthesis, folds the tracks to one root, and writes the container. The decoder is a synthesiser and a verifier — it reads the header, takes the cheapest route the file allows (a cache hit serves a proxy; a miss executes the instruction stream), reconstructs each frame under a determinism contract, and lets any party re-derive the whole from public inputs.

This document specifies both halves: the encoder pipeline and its coding decisions, the decoder pipeline and its synthesis path, the interface between the codec and whatever model does the generating, the content-addressed cache that makes recompute optional, the rate and quality control that trades size against editability, and the conformance a compliant implementation must meet. It is a *design*, written against a real substrate — the estate's .syn container, its content-addressed job cache, its evidence ledgers, and its generative-video pipeline — which the encoder and decoder compose rather than reinvent.

THE ONE INVARIANT THE CODEC EXISTS TO HOLD

A frame the decoder produces from an instruction must equal the frame the encoder saw when it wrote that instruction — or the codec must say it does not. Every design choice below serves this: the determinism key on each coding unit, the intra/predicted decision, the cache's *reproduced-versus-reused* grading, and the conformance rule that an unverifiable frame is reported SKIPPED, never silently passed. A codec that cannot guarantee the equality must be able to detect its own failure to; that detection is not an add-on, it is the product.

HOW THIS DESIGN IS QUALIFIED

This is a design document, and it marks what it rests on. [M2] Modeled — a component that already exists in the estate's media stack and that the codec composes: the .syn container and its Merkle work root, the content-addressed job cache, the provenance/attestation ledgers, the generative-video and non-destructive-assembly pipeline. [A1] Assumed — a component this

document *designs* and that is not yet built as described: the coding-unit bitstream and its intra/predicted decision, the entropy layer, the synthesis-engine interface as a codec contract, and the unified muxer/demuxer. Design parameters in figures and tables are targets and structure, never measured results.

H.SYN.EMA · codec design · companion to the format-specification whitepaper · prepared for the operator
substrate: the .syn container · the content-addressed job cache · the provenance ledgers · the
generative-video pipeline

Contents

1	What the Codec Does	4
1.1	Two planes, one bridge	4
1.2	The spine is determinism	5
2	The Encoder	6
2.1	Pipeline	6
2.2	The intra / predicted decision	6
2.2.1	What a P-Node stores	6
2.2.2	Drift is measured, not assumed	6
2.3	The encode loop	7
2.4	Muxing and the root fold	7
3	The Decoder	9
3.1	Pipeline	9
3.2	Random access without re-decoding the film	9
3.3	Rebuilding a predicted frame	10
3.4	When a frame will not reproduce	10
4	The Synthesis-Engine Interface	11
4.1	The contract	11
4.2	The determinism contract, made operational	12
5	The Cache	13
5.1	Two levels, one discipline	13
5.2	Reproduced is not reused	13
5.3	The cache is shared, and that is the point	14
6	Rate & Quality Control	15
6.1	The rate–drift trade	15
6.2	Presets	15
7	Conformance	17
7.1	Decoder profiles	17
7.2	Encoder duties	17
7.3	Golden vectors	17

8	The Honest Frontier	19
8.1	Status	19
	Colophon	21

What the Codec Does

A codec is an encoder and a decoder. For H.SYN.EMA the two are less symmetric than in pixel video, because the thing being coded is not a picture but the *procedure* that makes one. The encoder watches a production happen — seeds, prompts, deltas, renders — and writes down the procedure and its proof; the decoder replays the procedure, or, when it can, skips straight to a cached result. Between them sits the container, which is the procedure made durable.

1.1 Two planes, one bridge

Everything the codec touches lives on one of two planes, and the codec’s whole job is to keep them in exact correspondence.

plane	content
instruction	the SYN bitstream: coding units that <i>describe</i> how each frame is generated — seeds, prompts, deltas, parameter sets. Small, authored, the truth.
media	the frames, the proxy, the stems: the <i>output</i> of executing the instruction plane. Large, derived, a cache.

A pixel codec has only the media plane and tries to shrink it. This codec treats the media plane as *derivable* from the instruction plane and therefore optional to store — the encoder’s art is to make the instruction plane a faithful, minimal, reproducible description, and the decoder’s art is to reconstruct the media plane from it identically, or to prove it cannot.

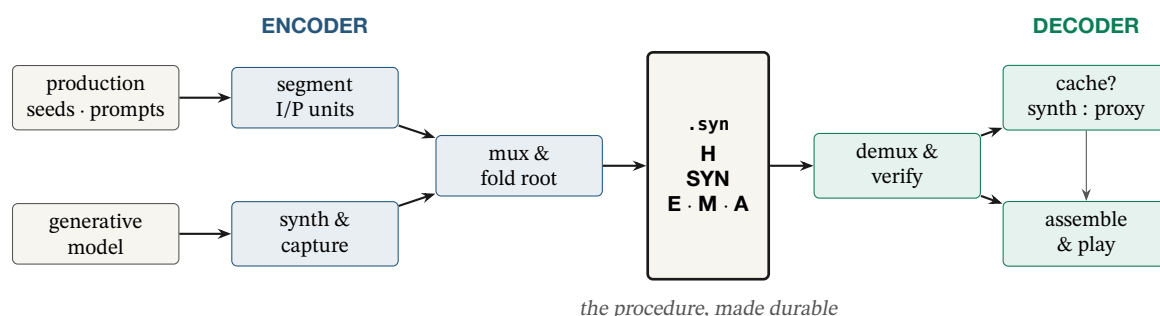


Figure 1.1. The codec, top to bottom. The encoder (*sky*) segments a production into intra and predicted coding units, drives the generative model to synthesise each and captures its provenance, then multiplexes the tracks and folds them to one root. The decoder (*seal*) demuxes and verifies, then takes the cheapest route — a cached render, or a fresh synthesis — and assembles the timeline. The container in the middle is the only thing that persists; both halves are stateless functions of it.

1.2 The spine is determinism

The two planes stay in correspondence only if executing an instruction reproduces the frame the encoder saw. That is not a property the codec can assume of a generative model — most are stochastic — so the codec *manufactures* it: every coding unit carries a canonical key over its engine, inputs, and parameters (§ the cache chapter), the encoder records the exact seed and sampler state that produced its frame, and the decoder re-executes under the same state. Where a model cannot be pinned to determinism, the codec does not pretend; it records the frame as a delivered payload rather than a reproducible instruction, and marks it so.

■ Encoder and decoder are inverses, or the file says they are not

The design's whole burden is that $\text{decode}(\text{encode}(x)) = x$ for the frames, and that any failure of that equality is *detectable* from the file alone. The chapters that follow are the two functions and the machinery — the synthesis interface, the cache, the rate control — that makes the equality hold in practice and legible when it does not.

The Encoder

The encoder turns a production into an instruction stream. It runs once, forward, over the sequence of frames a director has approved, and for each it makes one decision — intra or predicted — then drives the model, captures the proof, and appends a coding unit. When the sequence ends it folds the tracks and writes the header.

2.1 Pipeline

Six stages, each feeding the next; only SYNTHESISE touches a GPU.

#	stage	does
1	ingest	read the approved sequence and its parameter sets; emit an SPS
2	segment	decide intra vs predicted for this frame (below)
3	synthesise	drive the model; capture the exact seed and sampler state
4	capture	write a provenance record and machine attestation to E
5	code	entropy-code the unit's payload; append to SYN
6	mux	at end: build M and A, fold roots, write H

2.2 The intra / predicted decision

The one decision that shapes the whole file. An *intra* unit (I-Node) is self-contained and expensive in bytes but freely editable; a *predicted* unit (P-Node) is a small delta from an upstream anchor but welded to it. The encoder emits an I-Node when *any* of four conditions holds, and a P-Node otherwise:

2.2.1 What a P-Node stores

A predicted unit carries only the change from its reference I-Node: a prompt diff (the tokens added or removed), a temporal-delta or camera-motion matrix, and any control-signal vectors — never a fresh seed. The decoder reconstructs the P-Node's frame by conditioning the model on the reference's latent plus the delta, so a P-Node is meaningless without its anchor, which the codec enforces by refusing to place a P-Node before its referenced I-Node in stream order.

2.2.2 Drift is measured, not assumed

Drift is the encoder's estimate of how far a predicted chain has wandered from what a fresh intra synthesis would produce — the generative analogue of accumulated inter-frame error. The encoder computes it by periodically synthesising a shadow intra frame and comparing; when

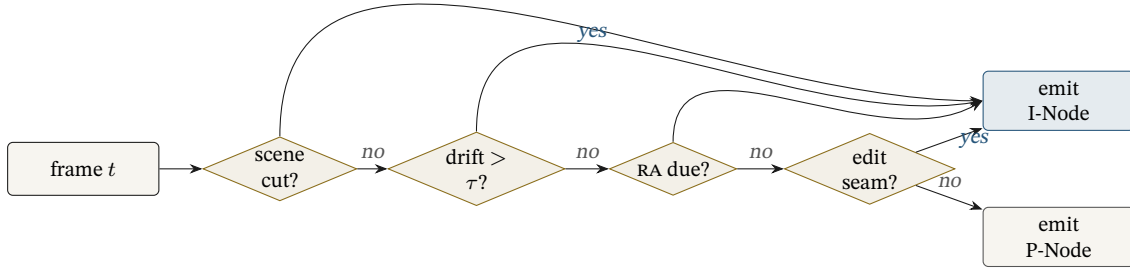


Figure 2.1. The coding decision, per frame. Any one condition forces an intra keyframe: a scene cut (the reference is useless across it), accumulated drift past a threshold τ (predicted error would compound), a random-access point coming due (so a player can seek), or an edit seam the director marked (so the shot stays independently re-cuttable). Absent all four, the frame is a small predicted delta. The four conditions are the encoder’s only tunable policy, and Chapter VI turns them into a rate/quality dial.

the distance crosses τ it forces an I-Node and resets the accumulator. The threshold is a quality knob, and it is honest: a codec that never checked drift would produce a stream that decodes to something plausible and *wrong*, which is the failure this measurement exists to catch.

2.3 The encode loop

```

sps ← emit_sps(model_digest, sampler, resolution)
anchor ← none
for t in approved_sequence:
    if intra_needed(t, drift, gop, seams):
        u ← I_node(seed=t.seed, prompt=t.prompt)
        anchor ← u; drift ← 0
    else:
        u ← P_node(ref=anchor, delta=diff(t, anchor))
        frame, state ← synth(u) // Chapter IV
        key ← canonical_key(u.engine, u.inputs, u.params)
        E.append(provenance(frame, u, key, attest()))
        SYN.append(entropy_code(u, key))
        drift ← update_drift(frame)
M ← dag_from(E.inputs) ; A ← compile(cut)
R ← fold(root(SYN), root(E), root(M), root(A))
write_header(magic="HSYN", R, directory)

```

2.4 Muxing and the root fold

When the sequence is coded, the encoder builds the two remaining tracks — M from the input-hash edges the provenance records already name, A from the director’s cut as pointers into that graph — then computes each track root and folds the four into the 32-byte *Root State Hash*. The header is written last, because the root cannot be known until every track is closed. A well-formed file is thus the encoder’s proof that it finished: a truncated encode has no valid root, and the decoder’s stage-5 fold check refuses it.

THE ENCODER CAPTURES, IT DOES NOT JUDGE

Every provenance record the encoder writes carries its own honest scope — a co-occurring-egress disclaimer, a machine-liveness-not-presence clause — because the encoder is recording *that* a machine synthesised a frame, never vouching for the frame's content. The codec's evidence is a notarised timestamp on a computation, and the encoder is careful to claim nothing more.

The Decoder

The decoder is a stateless function of the file. It reads the header, decides whether any synthesis is needed at all, and then either serves a cached render or walks the instruction stream. It is also the verifier: the same walk that reconstructs a frame re-derives the proof that the frame is what the instruction says.

3.1 Pipeline

#	stage	does
1	demux	parse H and the directory; locate each track by offset
2	verify	run the eight-stage check; refuse or mark SKIPPED
3	route	is the Root State Hash in cache? hit → serve A; miss → decode
4	decode	walk SYN; for each unit rebuild its inputs
5	synth	execute the unit under its determinism key; cache the frame
6	assemble	follow A's pointers into the decoded frames; play

3.2 Random access without re-decoding the film

Because every I-Node stands on nothing before it, the intra units are the stream's random-access points, exactly as keyframes are in pixel video. To present time t , the decoder seeks to the nearest preceding I-Node and decodes forward through its P-Nodes — never from the top of the film. The header's directory carries an index of I-Node offsets so the seek is a lookup, not a scan.

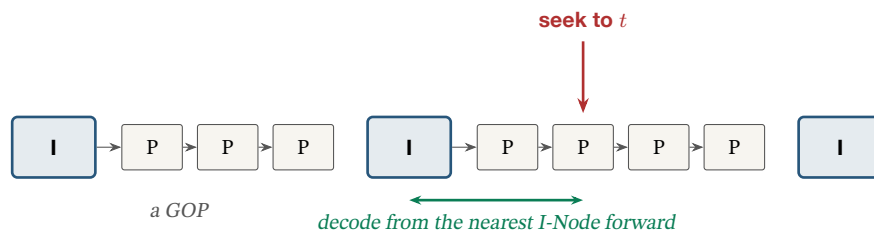


Figure 3.1. Seeking. To render time t the decoder jumps to the I-Node that opens t 's group and replays only its predicted units up to t — the two P-Nodes between the anchor and the target, not the hundreds before it. The intra cadence Chapter II chose is therefore also the seek granularity: denser I-Nodes cost bytes and buy faster random access.

3.3 Rebuilding a predicted frame

A P-Node carries a delta, not a seed, so the decoder must first hold its reference. It keeps the current I-Node's decoded latent resident and conditions the model on *that latent plus the P-Node's delta*; the result is the predicted frame. This is the one place the decoder carries state across units, and it is bounded: only the active GOP's anchor need be resident, so a decoder's working set is one latent, not the whole film.

```
demux(file); verify(file)    // refuse, or mark skipped
if cache.has(file.R): return serve(file.A.proxy, file.A.stems)
anchor_latent ← none
for u in file.SYN:
    key ← u.node_key
    if cache.has(key): frame ← cache.get(key)    // reproduced/reused
    elif u.kind == I:
        frame, anchor_latent ← synth(u.seed, u.prompt)
    else: // P
        frame ← synth_delta(anchor_latent, u.delta)
    cache.put(key, frame)
    if not reproduces(frame, u): report_skipped(u)
play(assemble(file.A, decoded_frames))
```

3.4 When a frame will not reproduce

If the decoder's synthesis of a unit does not match what the encoder recorded — a model version skew, a non-deterministic kernel, a corrupted delta — it does *not* substitute a plausible frame and move on. It reports the unit SKIPPED, names the reason, and lets the assembler fall back to the proxy for that span while flagging the gap. A film that plays with three unverifiable frames must never present as a film that verified.

THE DECODER NEVER UPGRADES COULD-NOT-CHECK TO FINE

The most dangerous decoder is the one that hides its own failure. This one treats an unreproducible frame the way the container treats a sealed asset: skipped, individually named, never counted as decoded-and-verified. A green playback is not a proof; the proof is the frames that actually reproduced, and the decoder reports exactly which those were.

The Synthesis-Engine Interface

The codec does not contain a generative model; it *drives* one across a narrow interface, so the same encoder and decoder work with any engine that can meet the contract. The interface is the seam that makes the format model-agnostic and the determinism guarantee possible at once.

4.1 The contract

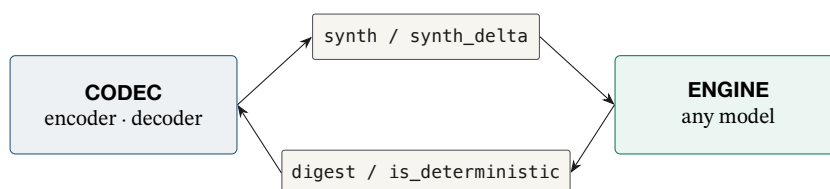
An engine offered to the codec must provide four operations and one promise.

operation	meaning
<code>digest()</code>	a stable hash of the model weights and architecture — what the SPS pins
<code>synth(seed, prompt)</code>	produce a frame and its latent from an intra unit
<code>synth_delta(latent, delta)</code>	produce a frame from a reference latent plus a predicted delta
<code>is_deterministic()</code>	whether identical inputs yield identical outputs, bit for bit

```

trait SynhEngine {
  fn digest(&self) -> Hash;
  fn synth(&self, seed: Seed, prompt: &Prompt)
    -> (Frame, Latent);
  fn synth_delta(&self, base: &Latent, d: &Delta)
    -> Frame;
  fn is_deterministic(&self) -> bool;
}

```



the only surface between the format and the model

Figure 4.1. The engine boundary. Everything model-specific lives to the right of four function signatures; everything format-specific lives to the left. The codec asks the engine to synthesise and to declare its identity and its determinism, and knows nothing else about it — so a new model is a new engine behind the same interface, and a file made with one model is decodable by anyone who holds that model’s weights, named by `digest()` in the SPS.

4.2 The determinism contract, made operational

`is_deterministic()` is the promise the whole codec leans on, and the codec verifies rather than trusts it. At encode time, an engine claiming determinism is spot-checked: a unit is synthesised twice and the frames compared; a mismatch downgrades the engine to non-deterministic for this file. Three regimes follow, and the codec is explicit about which it is in:

regime	how the codec codes it
deterministic	the unit stores the instruction only; the frame is reproducible and need not ship [A1]
seeded-stochastic	the unit pins the seed <i>and</i> the sampler state; reproducible given both [A1]
non-deterministic	the frame is stored as a delivered payload, marked not-reproducible; the instruction is provenance, not a recipe [M2]

The third regime is the honest floor: some engines cannot be pinned, and rather than pretend, the codec keeps the frame and says so, so a reader never mistakes a stored frame for a reproducible one. The reference engine in the estate — a diffusion video model driven through a rectified-flow sampler with a fixed seed — sits in the second regime, and the codec’s job is to record enough of its state that a second machine reproduces the frame bit for bit [\[M2\]](#).

■ The model is pinned, never assumed

A decoder that met a unit whose SPS named a model it did not hold would be entitled to guess — to substitute a different generator and produce a different film under the same root. The interface forbids it: `digest()` is checked before synthesis, and a mismatch is a refusal, not a fallback. The film is reproducible only by a party who holds the exact weights, and the codec would rather say “I cannot reproduce this” than quietly reproduce something else.

The Cache

The cache is what lets a film be shipped as instructions and still play instantly. It answers one question — *have I made this before?* — at two granularities, and it is content-addressed at both, so a hit is never trusted by name. This is the estate’s existing content-addressed job cache, generalised to two levels [M2].

5.1 Two levels, one discipline

level	key $\rightarrow$ value
whole render	the Root State Hash $R \rightarrow$ the complete decoded film / proxy
per unit	$\text{node_key} = \text{canonical}(\text{engine}, \text{inputs}, \text{params}) \rightarrow$ one frame

The decoder consults the render level first: if R names a completed render, it serves it and never touches the model. On a miss, it decodes unit by unit, and for each consults the per-unit level, synthesising only the frames it has not made before. A film that shares a hundred shots with a previous cut re-renders only what changed.

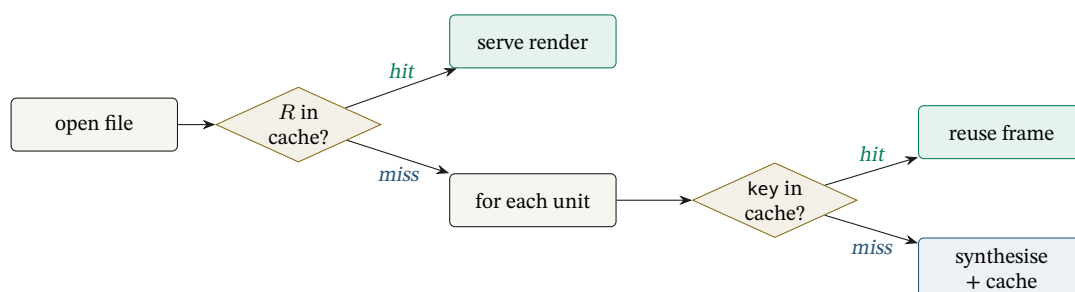


Figure 5.1. The two-level lookup. The render level short-circuits the whole pipeline when a film has already been decoded; the per-unit level short-circuits each frame. Both keys are content — the render by its Root State Hash, each frame by a canonical key over its exact computation — so a hit is a match of what would be computed, never of a filename or a timestamp.

5.2 Reproduced is not reused

A cache hit is graded, and the grade is recorded, because the two things it could mean are not the same. Reused means the codec served a frame it had stored under this key — a performance fact. Reproduced means the codec re-synthesised the frame and confirmed it matches — a correctness fact. A player takes reused frames and is fast; an auditor forces reproduction and is slow; and the file records which grade each frame carries, so “it played” and “it verified” are never conflated. Filling the cache from a reuse writes no new provenance record — a reuse is not a production, and must not masquerade as one.

5.3 The cache is shared, and that is the point

Because the keys are content, a cache can be shared across machines without trust: a GPU-rich machine decodes a film and publishes the frames under their keys; a GPU-poor machine, given the same file, finds those keys and plays without synthesising anything. The provenance travels in the file, not the cache, so the poor machine plays the rich machine's render and can still prove where every frame came from. The cache accelerates; it never becomes the evidence.

A NON-DETERMINISTIC ENGINE POISONS ONLY ITSELF

An engine the codec has flagged non-deterministic is never cached at the per-unit level — there is no stable key to store under, and a stored frame could not be trusted to recur. Its frames are carried as delivered payloads instead. The cache thus holds only what is reproducible, which is the only thing a content address can honestly name.

Rate & Quality Control

Pixel codecs trade bits against distortion. This codec trades bytes against *drift* and *editability*, and the single control is the intra cadence — how often the encoder spends a full keyframe instead of a cheap delta. Every knob in Chapter II is a way of setting that cadence.

6.1 The rate–drift trade

The analogue of rate–distortion is exact once the terms are named. *Rate* is bytes, and I-Nodes dominate it: a keyframe carries a full seed and prompt, a P-Node only a diff. *Drift* is the reconstruction error a predicted chain accumulates before its next anchor — the generative counterpart of inter-frame drift — and it *grows* with the distance since the last I-Node. Denser intra spends bytes to hold drift down; sparser intra saves bytes and lets drift climb, bounded only by the threshold τ that forces a reset.

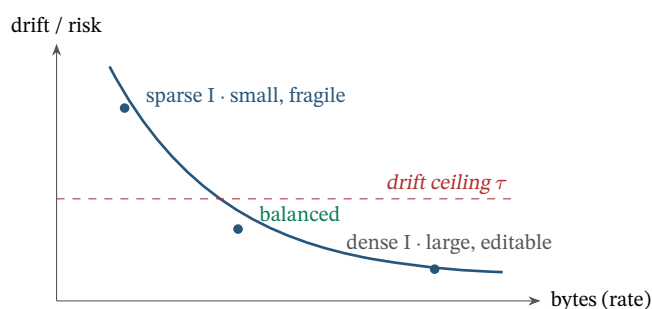


Figure 6.1. The trade, qualitatively (a design sketch, not a measurement). Spending more bytes on intra keyframes buys lower drift and faster random access; spending fewer buys a smaller file at rising reconstruction risk. The encoder’s job is to sit on the knee — the fewest I-Nodes that keep drift under the ceiling τ — and the operator’s presets move where that knee is set.

6.2 Presets

The four intra-forcing conditions collapse into named presets, so an operator chooses an intent, not four numbers:

preset	policy
ARCHIVAL	tight drift ceiling, short GOP, an I-Node at every marked seam — largest, most re-cuttable, seeks anywhere
MASTER	balanced ceiling and cadence — the default working format
DELIVERY	loose ceiling, long GOP, intra only at cuts — smallest, still faithful, coarse random access
DRAFT	synthesis off; carry the proxy and the instructions only — for review before a full render

A preset never changes what a frame *is*, only how often the encoder re-anchors and whether it renders at all. The same production encoded ARCHIVAL and DELIVERY decodes to the identical film wherever both carry a frame — the difference is byte size, seek granularity, and how much of the film each leaves independently editable.

■ **The dial is honest at both ends**

Turned toward DELIVERY, the codec must not let drift silently exceed τ to hit a size target — it forces an I-Node instead and grows the file, because a smaller file that decodes to the wrong frames is not a smaller file, it is a broken one. Turned toward ARCHIVAL, it must not pretend density it did not spend. The size a preset reports is the size it wrote, and the fidelity it claims is the fidelity the drift check confirmed.

Conformance

A format is only as good as the agreement between two implementations that never met. This chapter states what an encoder and a decoder must do to be conformant, and how that is checked — because a codec whose conformance is a matter of opinion is a codec that will one day disagree with itself.

7.1 Decoder profiles

Not every decoder needs to synthesise, but every decoder must be honest about what it did. Three profiles, in increasing strength; each must satisfy all the duties of the ones before it.

profile	must
PLAYER	run verify stages 1–5 (header, directory, trailer, roots, fold); serve the proxy; and honour SKIPPED — never present an unverified span as verified
VERIFIER	additionally run stages 6–8 (ledger, graph, synthesis reproduction) and report, per frame, <i>reproduced</i> or <i>skipped</i>
AUTHORING	additionally rebuild the M graph and re-fold roots after an edit, so a re-cut file is itself conformant

A PLAYER that finds a stage-4 root mismatch must refuse the file, not play it; a VERIFIER that cannot obtain the model named in the SPS must report the affected frames SKIPPED, not guess them. The single forbidden behaviour, at every profile, is upgrading could-not-check to fine.

7.2 Encoder duties

duty	meaning
close the root	write H only after every track root is final; a truncated encode has no valid <i>R</i>
check drift	force an I-Node before drift crosses τ ; never trade fidelity for a size target
pin the model	record a <code>digest()</code> a decoder can check; refuse to encode against an unpinable engine silently
grade honestly	mark each unit deterministic, seeded-stochastic, or delivered; never label a stored frame reproducible

7.3 Golden vectors

Conformance is not a promise; it is a set of files that a conformant implementation reproduces. The codec ships *golden vectors* — small `.syn` files with known roots — and a conformant en-

coder, given the same production and preset, must emit the same root, while a conformant verifier, given the vector, must reproduce every frame it marks reproduced and refuse every deliberately corrupted one. Three classes cover the surface:

- round-trip — a production encodes and decodes to a bit-identical film; the root is fixed and published.
- cross-decode — two independent decoders on the same vector agree, frame for frame, on which reproduced and which were skipped.
- tamper — a vector with one flipped byte in each track must be refused at the stage that owns that track, and the refusal named — a verifier that passes a tampered vector is non-conformant, which is the sabotage discipline the estate applies to every gate it trusts.

THE INTEROP GUARANTEE, STATED PLAINLY

Two conformant implementations, given the same production and preset, produce files with the same root; and any conformant verifier, given any conformant file, either reproduces exactly the frames the file claims are reproducible or names the ones it could not — and never confuses the two. That is the whole of what conformance buys, and it is enough: it makes the format a contract between strangers rather than a behaviour of one program.

The Honest Frontier

A codec design earns trust the way the format does — by naming what it cannot do. These limits are specific to the coder and decoder, distinct from the format’s own boundaries, and they are stated here so no implementer discovers them the hard way.

ENCODE COSTS WHAT THE RENDER COSTS

There is no free lunch on the encode side. To write a faithful instruction stream the encoder must actually synthesise each frame at least once — an I-Node’s keyframe, a P-Node’s predicted result, and the shadow intra frames the drift check consumes. Encoding is therefore as expensive as producing the film, plus the drift overhead; the codec’s economy is entirely on the *decode* side, where a cache hit makes re-render free. A codec that promised cheap encoding would be promising to write down frames it never computed.

A FILE IS RE-SYNTHESISABLE ONLY BY A HOLDER OF THE MODEL

The instruction stream reconstructs frames only for a decoder that holds the exact weights the SPS names. Without them the file still plays its proxy and still verifies its provenance and graph, but it cannot be re-rendered at full fidelity, and it cannot be re-cut into new shots that require fresh synthesis. Cross-model portability is not offered and cannot be: a different model is a different film. Preserving a work at full fidelity means preserving its weights beside it, and the codec pins the digest precisely so the substitution is refused rather than silently made.

THE ENTROPY WIN IS BOUNDED, AND THAT IS FINE

The instruction stream is already small — seeds, prompts, deltas — so entropy-coding it saves little in absolute terms; the format’s compression is not in shrinking the instructions but in *not shipping the frames at all*. An implementer chasing bitrate on the SYN track is optimising the wrong axis. The lever that matters is the intra cadence, which trades whole frames, not bits.

DETERMINISM IS BORROWED FROM THE ENGINE

The codec can pin a seed and a sampler state, but it cannot make a non-deterministic kernel deterministic. Where the engine will not reproduce, the codec falls back to storing the frame and marking it not-reproducible — an honest floor, not a fix. The reproducibility guarantee is exactly as strong as the weakest engine in the chain, and the file says which regime each frame is in rather than averaging them into a false whole.

8.1 Status

This is a design, and it is careful about that word. The coder and decoder are specified against a real substrate — the `.syn` container, the content-addressed job cache, the provenance ledgers, and a generative-video pipeline that already exists [M2] — and the parts this document adds, the coding-unit bitstream and its intra/predicted decision, the entropy layer, and the unified muxer/demuxer, are *designed here*, not yet built as one codec [A1]. Every figure’s numbers are

targets and structure; none is a benchmark. The reference skeleton that accompanies this design implements the interfaces and the container walk, and stubs the synthesis behind the engine trait — so the shape is executable even where the model is not yet wired.

Colophon

This design was set in the house style of the estate’s shelf — STIX Two Text and Helvetica Neue, the law-box idiom, native TikZ figures — as a companion to the H.SYN.EMA format-specification whitepaper. Build with `./build.sh` (XeLaTeX $\times 2$). It is an *architecture*: it specifies the encoder and decoder, their coding decisions, the synthesis interface, the cache, and conformance, in enough detail to implement and to test against, but it is not itself the running codec.

The substrate it composes

The codec is designed against parts that exist. The `.syn` container and its Merkle work root give the muxer its format and the fold its arithmetic; the estate’s content-addressed job cache — the one that asks its question before a subprocess starts and grades a hit *reproduced* versus *reused* — is the cache subsystem’s working referent; the provenance, witness, and attestation ledgers, with their inline anti-over-claim clauses, are what the encoder’s capture stage writes; and a generative-video pipeline with non-destructive assembly is what sits behind the synthesis interface. The codec’s contribution is the coding layer that binds them: the intra/predicted decision, the determinism-graded units, and the single walk that both reconstructs a film and verifies it.

One boundary held

Even written as a finished architecture, the design asserts no measurement it did not take. Byte layouts, the pipeline stages, the interface signatures, and the conformance rules are specification; the rate–drift curve and every figure’s numbers are structure and targets, drawn to show a shape, not to report a benchmark. That restraint is the codec’s own invariant turned on the document about it: *claim reproduction only where it can be checked, and name the frames where it cannot*.

A pixel codec hides what already exists; this one records what will, and proves it reproduces.

Encoding costs the render; decoding from cache costs nothing.

Encoder and decoder are inverses — or the file says, in the open, that they are not.

ONE CONTRACT · TWO FUNCTIONS · ONE ROOT · REPRODUCIBLE, OR NAMED.

H.SYN.EMA · codec design · Westwyrd Federated Instruments · prepared for the operator